

SPRINT 0 — DOCUMENTO DE PLANIFICACIÓN TÉCNICA

Proyecto: Rick & Morty Portal Finder

Autora:	Angie Plazas	Fecha inicio:	Lunes 03 de agosto de 2026
Presentado a:	Equipo de Desarrolladores EPAM Neoris	Punto control:	Miércoles 05 de agosto de 2026
Stack técnico:	JavaScript Vanilla (ES6+) · Web Components (Custom Elements + Shadow DOM)	Entrega final:	Viernes 07 de agosto de 2026

Índice

1. Objetivo general	10. Servicios
2. Alcance (Scope)	11. Componentes
3. Requerimientos	12. Roadmap (5 días laborales)
4. Historias de usuario	13. Checklist técnico
5. Casos de uso	14. Riesgos
6. Arquitectura	15. Estrategia de desarrollo
7. Modelo de datos	16. Estrategia de pruebas
8. Gestión de estado	17. Estrategia para la demo
9. Diseño de eventos	

1. Objetivo general

Construir **Rick & Morty Portal Finder**, una aplicación web que permite:

- Buscar personajes de la serie *Rick and Morty* por nombre.
- Filtrar dichos personajes por estado vital (`alive` , `dead` , `unknown`).
- Marcar/desmarcar personajes como favoritos, con persistencia entre recargas de página.
- Visualizar información estructurada de cada personaje (imagen, especie, estado, origen, ubicación, episodios).

El objetivo técnico central del proyecto es demostrar dominio de:

1. Arquitectura basada en componentes desacoplados (Web Components nativos).
2. Encapsulación real mediante Shadow DOM.
3. Comunicación unidireccional mediante Custom Events.
4. Manejo correcto de estados asíncronos (loading / success / error).
5. Persistencia de datos en el cliente (`localStorage`).
6. Buenas prácticas de ingeniería: principio de responsabilidad única (SRP), separación de responsabilidades y manejo seguro de datos externos.

2. Alcance (Scope)

Incluido en el MVP

- Búsqueda de personajes por nombre mediante consumo de API real.
- Filtro por estado (`alive` , `dead` , `unknown`) combinado con la búsqueda.
- Renderizado dinámico de tarjetas de personaje.
- Marcar/desmarcar como favorito desde la tarjeta.
- Lista de favoritos persistente en `localStorage` .
- Eliminación de favoritos desde la lista, mediante delegación de eventos.
- Manejo visual de tres estados de interfaz: cargando, éxito, error.
- Validación del input de búsqueda (vacío / caracteres inválidos).
- Semáforo visual de estado en el componente de filtros.

Fuera de alcance (decisión deliberada)

- Paginación de resultados.
- Vista de detalle en ruta independiente / enrutamiento SPA.
- Pruebas automatizadas (unitarias o end-to-end).
- Autenticación de usuarios.
- Frameworks de UI (React, Vue, Angular, Lit).
- Bundlers o herramientas de build (Webpack, Vite).
- Internacionalización y temas visuales.

Justificación de Alcance

Cada funcionalidad fuera de este alcance no aporta puntuación adicional según los criterios de evaluación y añade riesgo de introducir código no dominado completamente por la autora. Se prioriza un MVP sólido, íntegramente comprendido y defendible, sobre funcionalidades adicionales no esenciales.

3. Requerimientos

3.1 Requerimientos funcionales

ID	Requerimiento
RF-01	El usuario puede escribir un nombre y buscar personajes.
RF-02	El usuario puede filtrar por estado (alive/dead/unknown).
RF-03	El sistema valida que la búsqueda no esté vacía ni contenga caracteres especiales.
RF-04	El usuario puede marcar un personaje como favorito.
RF-05	El usuario puede eliminar un personaje de favoritos.
RF-06	Los favoritos persisten tras recargar la página.
RF-07	El sistema muestra tarjetas con imagen, nombre, especie, estado, origen, ubicación y cantidad de episodios.
RF-08	El sistema muestra un estado de carga mientras espera la respuesta de la API.
RF-09	El sistema muestra un mensaje de error si la API falla o no hay resultados.

3.2 Requerimientos no funcionales

ID	Requerimiento
RNF-01	Los estilos de cada componente deben estar aislados, sin filtrarse al DOM global.
RNF-02	El código debe ser legible: nombres descriptivos y funciones de responsabilidad acotada.
RNF-03	La inserción de múltiples nodos al DOM debe optimizarse mediante <code>DocumentFragment</code> .
RNF-04	La aplicación debe degradar con gracia ante datos incompletos de la API, mediante Optional Chaining.

3.3 Requerimientos técnicos

ID	Requerimiento
RT-01	Toda la interfaz se construye con Custom Elements (<code>customElements.define</code>).
RT-02	<code>CharacterCard</code> y <code>SearchFilters</code> utilizan Shadow DOM.
RT-03	La comunicación entre componentes es exclusivamente mediante Custom Events con <code>bubbles: true</code> y <code>composed: true</code> .
RT-04	El servicio de datos es una clase ES6 con método estático y URL base encapsulada mediante propiedad privada.
RT-05	Toda petición a la API utiliza <code>fetch</code> junto con <code>async/await</code> y bloques <code>try/catch</code> .
RT-06	Los favoritos se serializan y deserializan con <code>JSON.stringify</code> y <code>JSON.parse</code> .

3.4 Requerimientos arquitectónicos

ID	Requerimiento
RA-01	Cada componente respeta el principio de responsabilidad única (SRP).
RA-02	<code>RickMortyApp</code> es el único componente que administra el estado global de la aplicación.
RA-03	Ningún componente hijo manipula directamente a un componente hermano.
RA-04	El renderizado de listas utiliza <code>.map()</code> , <code>.filter()</code> y <code>DocumentFragment</code> .

4. Historias de usuario

HU-01

Como usuario

Quiero buscar personajes por nombre

Para encontrar rápidamente personajes específicos de Rick and Morty.

HU-02

Como usuario

Quiero filtrar personajes por su estado (vivo, muerto, desconocido)

Para reducir los resultados a lo que realmente me interesa.

HU-03

Como usuario

Quiero recibir una alerta visual si mi búsqueda es inválida

Para saber que debo corregir mi entrada antes de continuar.

HU-04

Como usuario

Quiero marcar un personaje como favorito desde su tarjeta

Para poder encontrarlo fácilmente después sin buscarlo de nuevo.

HU-05

Como usuario

Quiero que mis favoritos se mantengan guardados aunque recargue el navegador

Para no perder mi selección previa.

HU-06

Como usuario

Quiero eliminar un personaje de mi lista de favoritos

Para mantener mi lista organizada y relevante.

HU-07

Como usuario

Quiero ver un indicador de carga mientras se buscan los personajes

Para saber que el sistema está procesando mi solicitud.

HU-08

Como usuario

Quiero ver un mensaje claro si ocurre un error o no hay resultados

Para entender qué sucedió.

HU-09

Como usuario

Quiero ver información relevante de cada personaje

Para conocer sus detalles sin salir de la aplicación.

5. Casos de uso

CU-01: Buscar personajes por nombre y estado

Actor: Usuario | **Precondición:** La aplicación está cargada.

Flujo principal:

1. El usuario escribe un nombre en el input de búsqueda.
2. Opcionalmente selecciona un estado en el `<select>`.
3. El usuario envía el formulario.
4. `SearchFilters` valida la entrada.
5. Si es válida, dispara `search-submitted` con `{ query, status }`.
6. `RickMortyApp` cambia su estado interno a `loading`.
7. `RickMortyApp` invoca `RickAndMortyService.getCharacters(query, status)`.

8. Si la promesa resuelve con datos, el estado pasa a `success` y se renderizan las tarjetas.
9. Si la promesa falla o no hay resultados, el estado pasa a `error`.

Flujo alternativo: Si el input es inválido, `SearchFilters` muestra una alerta visual y no dispara el evento.

CU-02: Marcar personaje como favorito

Actor: Usuario | **Precondición:** Existe al menos un personaje renderizado.

Flujo principal:

1. El usuario hace clic en el botón de favorito dentro de `CharacterCard`.
2. `CharacterCard` dispara `toggle-favorite` con `{ characterId, characterData }`.
3. `RickMortyApp` actualiza el arreglo de favoritos en memoria.
4. `RickMortyApp` persiste el nuevo arreglo en `localStorage`.
5. `RickMortyApp` actualiza el estado visual reflejado en `FavoritesList`.

CU-03: Eliminar un favorito desde la lista de favoritos

Actor: Usuario | **Precondición:** Existe al menos un favorito guardado.

Flujo principal:

1. El usuario hace clic en "eliminar" sobre un ítem dentro de `FavoritesList`.
2. Mediante delegación de eventos, un único listener en el contenedor captura el clic.
3. `FavoritesList` identifica el ítem eliminado.
4. `FavoritesList` dispara `favorites-updated` con el nuevo arreglo.
5. `RickMortyApp` actualiza su estado y persiste en `localStorage`.

CU-04: Recuperar favoritos al recargar la página

Actor: Sistema | **Precondición:** El usuario había guardado favoritos previamente.

Flujo principal:

1. Al montarse `RickMortyApp`, se lee `localStorage`.
2. Se deserializa con `JSON.parse`.
3. Si existen datos válidos, se cargan al estado `favorites`.
4. Se renderiza `FavoritesList` con esos datos.

Flujo alternativo: Si `localStorage` está vacío o corrupto, se inicializa `favorites` como arreglo vacío, sin lanzar error.

6. Arquitectura

6.1 Organización de carpetas

```
rick-morty-portal/
├── index.html
├── style.css
├── src/
│   ├── services/
│   │   └── RickAndMortyService.js
│   └── components/
│       ├── RickMortyApp.js
│       └── SearchFilters.js
```

```

├── FavoritesList.js
├── CharacterCard.js

```

Esta estructura separa por **tipo de responsabilidad**: `services/` contiene lógica de acceso a datos (sin DOM), y `components/` contiene lógica de presentación e interacción.

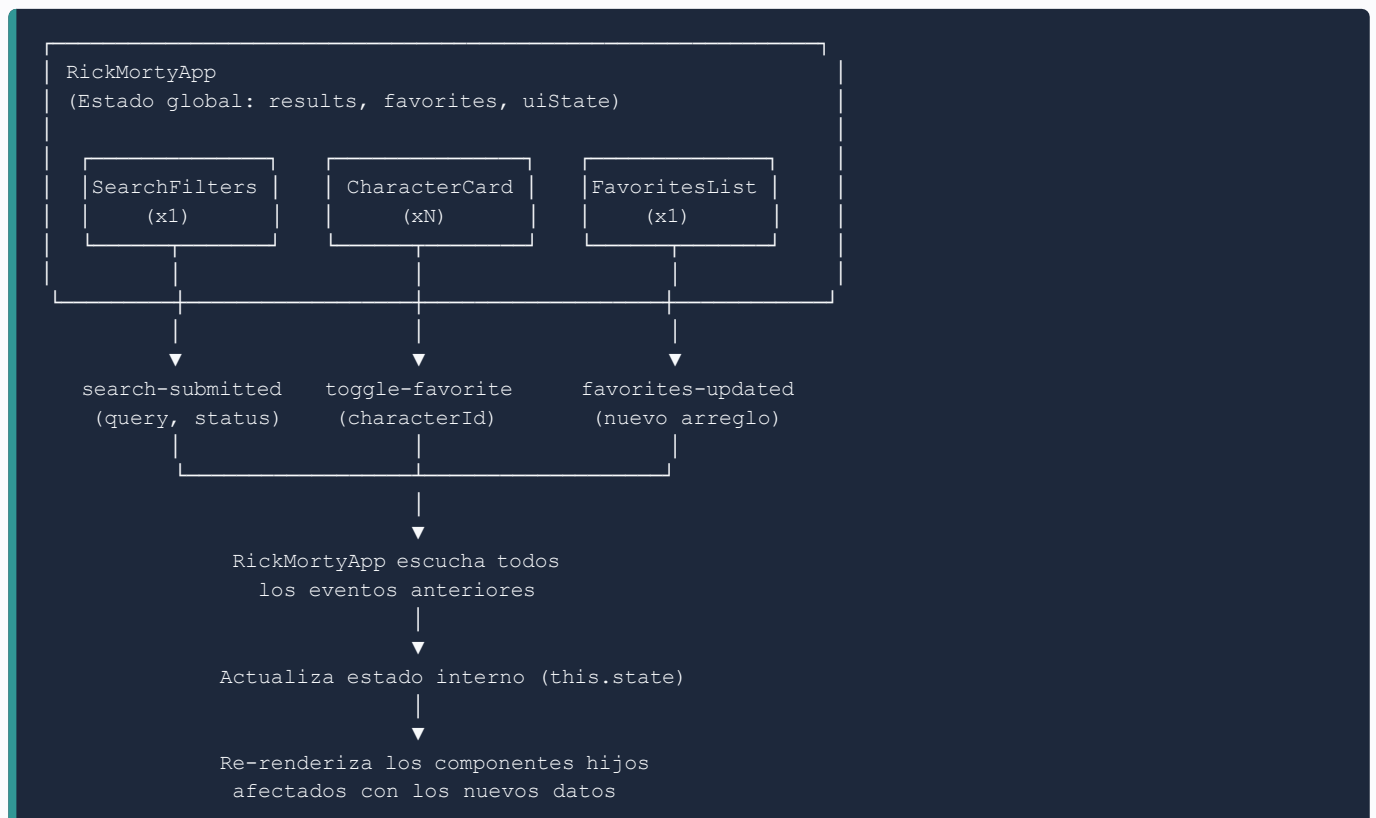
6.2 Responsabilidad de cada archivo

Archivo	Responsabilidad única
<code>RickAndMortyService.js</code>	Encapsular toda la comunicación con la API externa.
<code>RickMortyApp.js</code>	Orquestar el estado global de la aplicación.
<code>SearchFilters.js</code>	Capturar y validar el input del usuario, emitir la intención de búsqueda.
<code>CharacterCard.js</code>	Representar visualmente un personaje y notificar intención de favorito.
<code>FavoritesList.js</code>	Representar la colección de favoritos y permitir su eliminación.

6.3 Principio de Responsabilidad Única (SRP)

Cada componente cumple una única función dentro del sistema. Ningún componente combina lógica de presentación con lógica de persistencia o de acceso a datos. Por ejemplo, `CharacterCard` únicamente **notifica** la intención de marcar/desmarcar un favorito; es `RickMortyApp` quien decide cómo procesar y almacenar esa intención.

6.4 Flujo completo de datos



Este es un flujo unidireccional de datos: el estado vive en el componente raíz (`RickMortyApp`), los eventos suben mediante *bubbling*, y los datos bajan hacia los hijos mediante atributos y propiedades.

6.5 Comunicación mediante Custom Events

Toda comunicación entre componentes se realiza exclusivamente mediante `CustomEvent`, configurados de la siguiente forma:

```
new CustomEvent('nombre-evento', {
  detail: { /* payload */ },
  bubbles: true,
  composed: true
});
```

- `bubbles: true` permite que el evento ascienda por el árbol del DOM hasta ser capturado por `RickMortyApp`.
- `composed: true` permite que el evento atraviese el límite del Shadow DOM, requisito indispensable dado que `CharacterCard` y `SearchFilters` encapsulan su contenido mediante `attachShadow`.

Se optó por Custom Events —en lugar de callbacks pasados como propiedades JavaScript— por ser el mecanismo estándar y nativo de comunicación en el DOM, compatible con la filosofía de Web Components como elementos autónomos y reutilizables.

7. Modelo de datos

7.1 Character

Estructura de datos normalizada, utilizada como objeto plano (no como clase), dado que no requiere comportamiento propio, únicamente transporte de información entre el servicio y los componentes de presentación:

```
{
  id: Number,
  name: String,
  status: "Alive" | "Dead" | "unknown",
  species: String,
  image: String,
  origin: String,
  location: String,
  episodeCount: Number
}
```

7.2 Favorite

Subconjunto reducido del modelo Character, conteniendo únicamente la información necesaria para su representación en `FavoritesList`:

```
{
  id: Number,
  name: String,
  image: String
}
```

Esta reducción responde al principio de almacenar exclusivamente la información mínima necesaria, considerando además las limitaciones de espacio de `localStorage`.

7.3 Estados de la aplicación

El estado de interfaz se modela mediante un único campo discriminante (status), evitando el uso de múltiples banderas booleanas independientes, lo cual previene la posibilidad de estados contradictorios o imposibles:

```
{
  status: "idle" | "loading" | "success" | "error",
  data: [],
  errorMessage: ""
}
```

8. Gestión de estado

Componente	Estado que administra	Naturaleza
<code>RickMortyApp</code>	<code>{ status, characters, favorites, errorMessage }</code>	Estado global, único punto de verdad de la aplicación.
<code>SearchFilters</code>	<code>{ inputValue, selectedStatus }</code>	Estado local y temporal, limitado al ciclo de vida del propio formulario.
<code>CharacterCard</code>	Ninguno relevante	Componente de presentación pura; recibe todo mediante propiedades/atributos.
<code>FavoritesList</code>	Ninguno propio	Refleja exclusivamente los datos recibidos desde <code>RickMortyApp</code> .

Principio aplicado: cualquier dato requerido por más de un componente, o que deba persistir más allá del ciclo de vida de un componente individual, se centraliza en `RickMortyApp`.

9. Diseño de eventos

Evento	Disparado por	Escuchado por	Payload (event.detail)	Propósito
<code>search-submitted</code>	<code>SearchFilters</code>	<code>RickMortyApp</code>	<code>{ query: string, status: string }</code>	Notificar una intención de búsqueda válida.
<code>toggle-favorite</code>	<code>CharacterCard</code>	<code>RickMortyApp</code>	<code>{ characterId: number, characterData: object }</code>	Notificar intención de agregar/quitar un favorito.
<code>favorites-updated</code>	<code>FavoritesList</code>	<code>RickMortyApp</code>	<code>{ favorites: Array }</code>	Notificar un cambio en la lista de favoritos.

10. Servicios

RickAndMortyService

Responsabilidades:

- Encapsular la URL base de la API mediante una propiedad privada (`#BASE_URL`).
- Construir la URL de consulta según los parámetros recibidos (`name` , `status`).
- Ejecutar la petición mediante `fetch` con sintaxis `async/await`.
- Manejar errores de red o de respuesta HTTP mediante `try/catch`.
- Retornar los datos en formato JSON o lanzar un error controlado.

Fuera de sus responsabilidades:

- No interactúa con el DOM.
- No decide cómo se muestran los errores visualmente; únicamente los propaga.
- No transforma los datos a la forma final requerida por la interfaz.

Firma de la clase:

```
export class RickAndMortyService {
  static #BASE_URL = 'https://rickandmortyapi.com/api/character';

  static async getCharacters(name = '', status = '') {
```



```
// Construcción de query params, fetch, manejo de errores
}
}
```

El uso de métodos estáticos responde a que el servicio no mantiene estado propio entre llamadas. El uso de una propiedad privada (`#BASE_URL`) garantiza encapsulación real a nivel de motor de JavaScript, impidiendo el acceso o modificación externa de la URL base.

11. Componentes

11.1 RickMortyApp

Aspecto	Detalle
Responsabilidad	Orquestrar el estado global y coordinar la comunicación entre componentes.
Entradas	Ninguna (raíz de la aplicación).
Eventos escuchados	<code>search-submitted</code> , <code>toggle-favorite</code> , <code>favorites-updated</code>
Estado interno	<code>{ status, characters, favorites, errorMessage }</code>
Ciclo de vida	<code>connectedCallback</code>

11.2 SearchFilters

Aspecto	Detalle
Responsabilidad	Capturar y validar la búsqueda del usuario; mostrar el semáforo de estado.
Eventos disparados	<code>search-submitted</code>
Estado interno	<code>{ inputValue, selectedStatus }</code>
Shadow DOM	Sí
Ciclo de vida	<code>connectedCallback</code>

11.3 CharacterCard

Aspecto	Detalle
Responsabilidad	Representar visualmente un personaje y notificar intención de favorito.
Entradas	Propiedades <code>character</code> , <code>isFavorite</code>
Eventos disparados	<code>toggle-favorite</code>
Estado interno	Ninguno relevante (presentación pura).
Shadow DOM	Sí
Ciclo de vida	<code>connectedCallback</code>

11.4 FavoritesList

Aspecto	Detalle
Responsabilidad	Representar la colección de favoritos y permitir su eliminación.
Entradas	Propiedad <code>favorites</code>

Eventos disparados	<code>favorites-updated</code>
Estado interno	Ninguno propio.
Ciclo de vida	<code>connectedCallback</code>

12. Roadmap (5 días laborales)

Lunes 03 de agosto — Fundamentos: Servicio y esqueleto de componentes

Objetivo: Capa de datos funcional y componentes registrados.

Tareas: Implementación completa de `RickAndMortyService` ; verificación aislada por consola; registro de los cuatro Custom Elements; montaje inicial en `index.html` .

Entregable: Servicio verificado y componentes registrados sin errores en consola.

Martes 04 de agosto — Estado global y flujo de eventos

Objetivo: Flujo de datos funcional de punta a punta.

Tareas: Estado interno de `RickMortyApp` ; implementación de `SearchFilters` con validación; conexión completa del flujo `search-submitted` → estados loading/success/error; renderizado dinámico inicial con `DocumentFragment` .

Entregable: Búsqueda funcional visible en pantalla (sin estilos definitivos).

Miércoles 05 de agosto — Punto de control

Objetivo: Demostrar el flujo arquitectónico completo funcionando.

A presentar en el punto de control:

- Flujo de búsqueda completo operativo (loading → success/error).
- Arquitectura de eventos explicada y verificable en consola/DevTools.
- Avance de `CharacterCard` con Shadow DOM.
- Justificación de decisiones arquitectónicas tomadas hasta el momento.

Tareas del día: Implementación de `CharacterCard` con Shadow DOM y extracción segura de datos anidados (Optional Chaining); conexión del flujo de favoritos (`toggle-favorite`) con persistencia inicial en `localStorage` .

Entregable: Aplicación con búsqueda, visualización de tarjetas reales y favoritos funcionales, lista para revisión de avance.

Jueves 06 de agosto — Favoritos completos, delegación de eventos y estilos

Objetivo: Completar interacciones secundarias y dar identidad visual a la aplicación.

Tareas: Implementación de `FavoritesList` con delegación de eventos; semáforo visual de estado en `SearchFilters` ; estilos globales (`style.css`) y estilos encapsulados por componente.

Entregable: Aplicación visual y funcionalmente completa.

Viernes 07 de agosto — QA, refinamiento y preparación de la defensa

Objetivo: Verificación exhaustiva y preparación para la presentación final.

Tareas: Ejecución de la checklist técnica completa; pruebas de casos límite; limpieza de código y consola; preparación de la demostración.

Entregable: Proyecto finalizado, sin errores conocidos, listo para presentación ante el equipo evaluador.

13. Checklist técnico

Arquitectura y Modularidad

- ☐ Cada componente en su propio archivo.
- ☐ Todos los componentes heredan de HTMLElement.
- ☐ Todos los componentes usan customElements.define.
- ☐ RickMortyApp es el único con estado global.
- ☐ Ningún componente modifica directamente a un hermano.
- ☐ RickAndMortyService implementado como clase ES6.

Encapsulación y Shadow DOM

- ☐ CharacterCard usa attachShadow({ mode: 'open' }).
- ☐ SearchFilters usa attachShadow({ mode: 'open' }).
- ☐ Estilos internos no se filtran al DOM global.
- ☐ Estilos globales no afectan el interior de los Shadow DOM.

Comunicación e Interacción

- ☐ Los tres eventos personalizados usan bubbles: true, composed: true.
- ☐ RickMortyApp escucha correctamente los tres eventos.
- ☐ Delegación de eventos implementada en FavoritesList.

Robustez y Manejo de Datos

- ☐ Estados loading/success/error correctamente gestionados.
- ☐ try/catch implementado en el servicio.
- ☐ Optional Chaining utilizado en propiedades anidadas.
- ☐ localStorage con JSON.stringify/JSON.parse.
- ☐ .map() y .filter() utilizados según corresponda.
- ☐ Validación de input (vacío / caracteres especiales).

Buenas Prácticas de Código

- ☐ Nombres descriptivos.
- ☐ Comentarios solo donde aportan valor.
- ☐ DocumentFragment utilizado antes de inserciones múltiples.
- ☐ Sin console.log de depuración remanentes.
- ☐ console.table() tras búsquedas exitosas.
- ☐ console.error() en manejo de errores.

Requisitos puntuales del enunciado

- ☐ Semáforo visual funcional (verde/rojo/gris).
- ☐ #BASE_URL como propiedad privada.
- ☐ getCharacters como método estático.
- ☐ Extracción completa de datos en CharacterCard.

14. Riesgos

Riesgo	Mitigación
Eventos no propagados correctamente desde el Shadow DOM	Verificar cada evento con <code>console.log</code> en el listener antes de construir lógica adicional.
Acoplamiento indebido entre componentes hermanos	Revisión de cada componente antes de considerarlo finalizado.
<code>localStorage</code> corrupto o vacío	Manejo con <code>try/catch</code> y valor por defecto (<code>[]</code>).
Exceso de tiempo invertido en estilos	Roadmap prioriza lógica funcional sobre estética.
Caída o latencia de la API externa durante la demo	Preparar evidencia adicional (capturas/video) como respaldo.
Validación de input demasiado restrictiva	Pruebas con casos reales antes de finalizar la validación.

15. Estrategia de desarrollo

El desarrollo sigue un orden deliberado:

1. Servicio de datos primero, para contar con la forma real de los datos antes de construir interfaz sobre ellos.
2. Estado global y flujo de eventos, priorizando la correctitud del flujo de datos sobre la apariencia visual.
3. Componentización visual con Shadow DOM, una vez validado el flujo de datos.
4. Interacciones secundarias (delegación de eventos, semáforo visual).
5. Verificación y pulido final.

Cada etapa cuenta con una condición de finalización verificable antes de avanzar a la siguiente, evitando la acumulación de deuda técnica no detectada.

16. Estrategia de pruebas (checklist manual)

- Verificación de búsquedas válidas e inválidas contra el servicio.
- Verificación de validación de formulario (vacío, caracteres especiales).
- Verificación visual de los tres estados de interfaz (loading/success/error).
- Verificación de Shadow DOM mediante DevTools.
- Verificación de degradación segura ante datos incompletos de la API.
- Verificación de persistencia de favoritos tras recarga de página.
- Verificación de eliminación de favoritos mediante delegación de eventos.
- Prueba de integración completa del flujo de punta a punta.

17. Estrategia para la demo

Orden de presentación:

1. Demostración funcional de la aplicación (búsqueda, filtro, favoritos, persistencia).
2. Explicación de la arquitectura general mediante el diagrama de flujo de datos.
3. Explicación de un flujo de evento completo, de punta a punta.
4. Revisión del servicio de datos y su encapsulación.

5. Inspección en vivo del Shadow DOM mediante DevTools.
6. Demostración de manejo de errores provocando una búsqueda sin resultados.